

Tools in LangChain

Tools are one of the most important concepts in LangChain because they allow LLMs to interact with the outside world.

Without tools:

LLMs can only generate text

With tools:

LLMs can:

- Search the web
 - Use calculators
 - Access databases
 - Read files
 - Call APIs
 - Execute code
-

1. Intro

LLMs are powerful, but they have limitations.

For example:

- They cannot perform live Google searches by themselves
- They cannot access real-time data automatically
- They cannot calculate accurately every time
- They cannot directly interact with external systems

Tools solve this problem.

2. Overview

In LangChain:

Tools = abilities/functions that an AI agent can use

Simple Analogy

Think of the LLM as a human brain

And tools are:

- Calculator
- Internet
- Database
- Weather app

The brain decides:

“Which tool should I use?”

3. What is a Tool?

Simple Definition

A Tool is:

A function that an LLM or Agent can call to perform a specific task.

Core Idea

Instead of answering everything from memory:

The AI can use external helpers.

Example Without Tool

User:

“What is 923×481 ?”

LLM may calculate incorrectly

Example With Calculator Tool

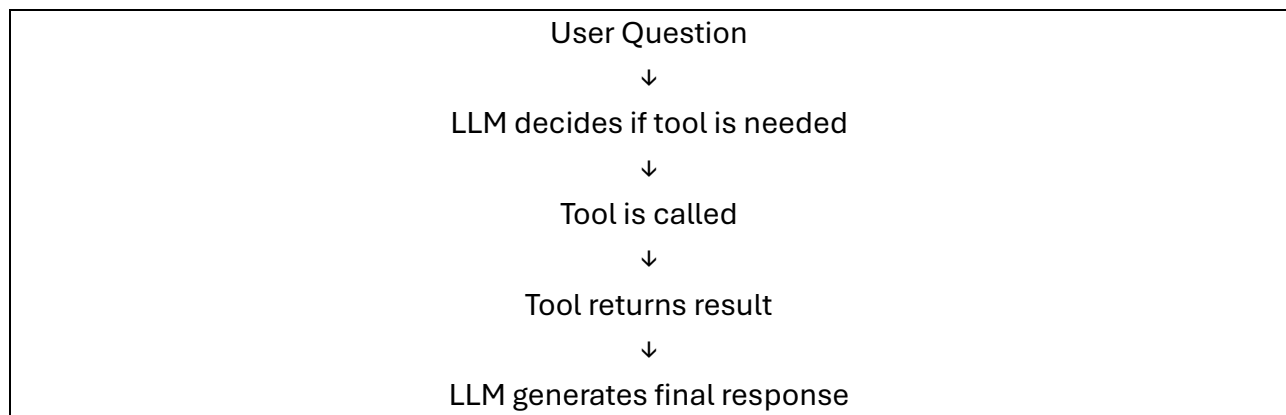
AI:

1. Uses calculator tool
2. Gets exact answer
3. Returns accurate response

Much better

How Tools Work

Basic Workflow



Real Example

User:

“What’s the weather in Islamabad?”

AI Process:

1. Detects need for real-time information
 2. Uses weather API tool
 3. Gets latest weather
 4. Responds to user
-

Types of Tasks Tools Can Handle

Task	Example Tool
Math	Calculator
Search	Google/Wikipedia
Databases	SQL Tool
APIs	Weather API
File Access	PDF Reader
Code Execution	Python Tool

4. Built-in Tools

LangChain provides many ready-made tools.

Common Built-in Tools**1. Wikipedia Tool**

Used to retrieve information from Wikipedia.

Example:

```
from langchain.tools import WikipediaQueryRun
```

Use Case:

- General knowledge
 - Educational bots
-

2. Python REPL Tool

Allows AI to execute Python code.

Example:

```
from langchain.tools import PythonREPLTool
```

Use Case:

- Calculations
- Data analysis

3. Requests Tool

Used to call APIs or websites.

Example:

```
from langchain.tools import RequestsGetTool
```

Use Case:

- Web APIs
- Live information

4. SQL Database Tool

Allows querying databases.

Example:

```
from langchain.tools.sql_database.tool import QuerySQLDataBaseTool
```

Use Case:

- Business dashboards
- Data assistants

Why Built-in Tools Are Useful

They:

Save development time

Are pre-tested

Easily integrate with agents

5. Custom Tools

Sometimes built-in tools are not enough.

You may need:

- Company-specific logic
- Custom API
- Special calculations

Then you create:

Custom Tools

Example Scenario

Suppose you own a store.

You want AI to:

- Check product stock
- Track orders

This requires your own business logic.

6. Create Custom Tools

Basic Idea

A custom tool is usually:

A Python function converted into a tool

Example

```
def multiply(a, b):
```

```
    return a * b
```

Convert to tool:

```
from langchain.tools import tool
```

```
@tool
```

```
def multiply(a: int, b: int) -> int:
```

```
    return a * b
```

What Happens?

Now the AI can use:

```
multiply(5, 4)
```

Output:

```
20
```

Why Decorator @tool Is Important

It tells LangChain:

“This function should behave like a tool.”

Important Components of a Tool

Component	Purpose
Name	Tool identifier
Description	Explains when to use tool

Function	Actual logic
----------	--------------

Why Description Matters

The LLM reads the description to decide:

“Should I use this tool?”

Bad Description:

“Does stuff”

Good Description:

“Useful for multiplying two integers”

Tool Input & Output

Input:

Data passed into tool

Output:

Result returned by tool

Example:

Input → (5, 10)

Output → 50

7. Toolkits

What is a Toolkit?

A Toolkit is:

A collection/group of related tools

Simple Analogy

Toolkit = toolbox 

Inside toolbox:

- Hammer
 - Screwdriver
 - Wrench
-

Example in LangChain

SQL Toolkit may contain:

- Query tool
-

- Schema tool
- Table inspection tool

Why Toolkits Are Useful

Instead of adding tools one-by-one:

You get a complete package

Common Toolkits

Toolkit	Purpose
SQL Toolkit	Database operations
Gmail Toolkit	Email operations
Vector Store Toolkit	Retrieval operations

Tools vs Chains vs Runnables

This is VERY important.

Concept	Purpose
Tool	External capability
Chain	Workflow/pipeline
Runnable	Standard processing unit
Agent	Decision-maker using tools

Big Picture Understanding

Without Tools

LLM:

Only text generation

No real-world interaction

With Tools

LLM:

Can access external systems

Can retrieve live data

Can automate tasks

Real-World Example

AI Travel Assistant

User:

“Find cheap flights and summarize weather in Dubai”

AI Process:

1. Uses flight API tool
 2. Uses weather API tool
 3. Combines results
 4. Generates final response
-

This is impossible without tools.

Important Concepts to Remember

1. Tools extend LLM capabilities

They connect AI to the outside world.

2. Tools are usually functions

Often Python functions wrapped as tools.

3. Agents use tools intelligently

Agents decide:

- Which tool to use
 - When to use it
-

4. Descriptions are critical

Good descriptions improve tool selection.

Common Mistakes

Poor tool descriptions

LLM gets confused.

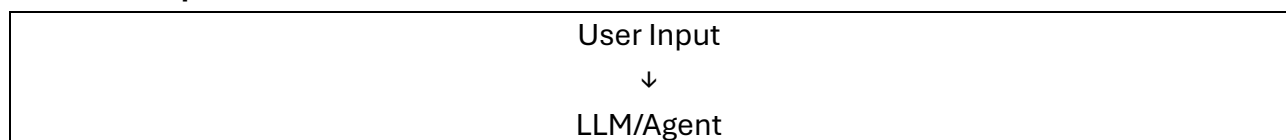
Too many unnecessary tools

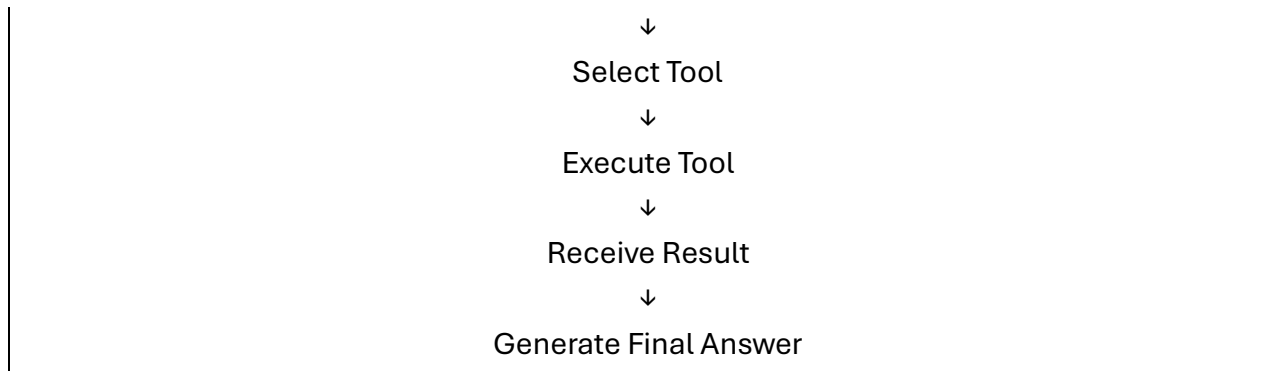
Agent struggles to choose.

Wrong tool outputs

Can break workflows.

Full Conceptual Flow





Final Summary

- Tools allow LLMs to interact with external systems
- Built-in tools are pre-made helpers
- Custom tools allow personalized functionality
- Toolkits are collections of related tools
- Tools are essential for:
 - Agents
 - Automation
 - Real-world AI systems

👉 Tools give AI “abilities” beyond normal text generation.